DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ABES Engineering College, Ghaziabad

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: MarketLab: Historical Stock Market Simulator with Machine Learning
Project Code: P_195
Format reference: IEEE Std 830-1998

SUBMITTED BY
Alok Srivastava
Roll Number: 2400320100149

UNDER THE GUIDANCE OF
Mr. Faizan
Department of Computer Science and Engineering

Degree: Bachelor of Technology (B.Tech)
Year: Third Year
Semester: Fifth Semester
Academic Year: 2026-27

Project: P_195
Document: Software Requirements Specification
Date of Submission: 1 October 2026

Document Revision and Review History

Version	Date	Description	Prepared By	Review Status
1.0	1 October 2026	Project-specific SRS prepared for academic submission	Alok Srivastava	Pending academic review

> Document note: This report follows the section structure of the supplied B.Tech SRS template. It describes the implemented project and identifies current limits. Performance and security statements are not presented as independently certified results.

1. Introduction

1.1 Purpose

This document describes the requirements and current scope of MarketLab. It explains how a learner can study historical Indian stock prices, place virtual orders, record a prediction and review the outcome. It is intended for the project guide, academic evaluators and the student developer. It also provides a reference for future changes to the application.

1.2 Scope of the System

Beginners often judge a trading decision only after they know the result. MarketLab provides a practical way to record the decision first and then reveal what happened. The application replays historical prices from January 2019 to December 2021 and allows users to practise with virtual money.

- In scope: account registration and login, historical data import, simulation setup, stock research, virtual trading, automatic model training, decision journaling, portfolio tracking, learning lessons, a 20-day challenge and results comparison.
- Outside the current scope: real money trading, live brokerage orders, intraday execution, derivatives, short selling, leverage, an administrator dashboard and downloadable PDF or spreadsheet reports.
- Expected benefits: learners can practise without risking money, compare their predictions with recorded outcomes and understand the difference between a correct direction forecast and a profitable trade. These are learning goals, rather than measured claims of financial improvement.

1.3 Definitions, Acronyms, and Abbreviations

Category	Term	Meaning
Documentation	SRS	Software Requirements Specification
Interface	API / REST	Application endpoints used to exchange requests and responses
Security	JWT	Signed token used to identify a logged-in user
Market data	OHLCV	Open, high, low, close and trading volume
Trading	P&L	Profit and loss, either realised or unrealised
Accounting	Equity	Cash plus the latest value of open holdings
ML	Logistic regression	Classifier that estimates the probability of an Up outcome
Evaluation	Holdout / baseline	Later test examples / a simple comparison prediction
Risk	Drawdown	Decline in portfolio value from an earlier peak

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications. Used as the structural reference supplied with the college template.
- Ian Sommerville, Software Engineering, 10th edition, Pearson. General reference for software requirements and system design.
- Project source and README: https://github.com/GuTS805/MarketLab. Implementation references: backend/app/main.py, backend/app/ml.py, backend/app/models.py and backend/import_market.py. Dataset: vishnun0027/indian-market-historical-ohlcv, revision fe783082c7ec68847ecd0d0b4e1ca06364507e5f.

1.5 Document Overview

Section 2 describes the application, users and operating environment. Section 3 lists the functional requirements. Section 4 explains the interfaces. Section 5 records quality requirements and verification methods. Section 6 presents the main design models. Section 7 provides space for academic review and signatures.

2. Overall Description

2.1 Product Perspective

MarketLab is a browser-based application with a Next.js presentation layer, a FastAPI application layer and a SQLite data layer accessed through SQLAlchemy. Historical prices are imported before a demonstration. The browser requests only information available through the current simulation date. The same backend handles trading, model evaluation and journal outcomes.

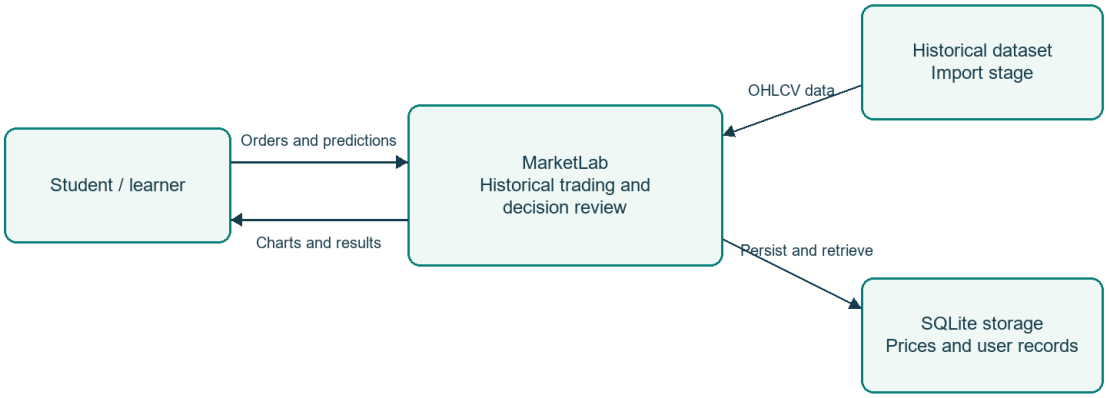


Figure 1. System context for a learner using MarketLab.

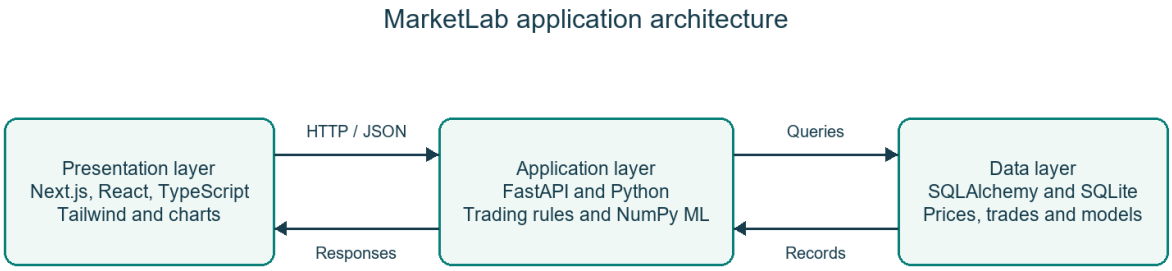


Figure 2. Presentation, application and data layers.

2.2 User Classes and Characteristics

The application supports visitors and registered learners. The developer maintains the local installation and imports the dataset through a command-line script. There is no separate administrator account or administrative web console in the current application.

User / Role	Experience	Responsibilities and Access
Registered learner	Basic to intermediate	Create simulations, research stocks, place virtual orders, record predictions and review personal results.
Visitor	Basic	View the login page and register or log in. Protected simulation data requires authentication.
Developer / maintainer	Technical	Install dependencies, run the importer, maintain the database and verify code. This is an operating responsibility, not an in-app admin role.

2.3 Operating Environment

The project is intended for local development and academic demonstration. The current setup uses Windows, Python, Node.js and a modern browser.

- Client: a modern browser with JavaScript enabled. The interface adapts to desktop and mobile widths. A keyboard or touch input is sufficient for ordinary use.
- Runtime: Python 3.10 or later for the backend, subject to dependency compatibility, and Node.js 20.9 or later for Next.js 16. The application has been tested locally with Python 3.13. GPU hardware is not required for the NumPy model.
- Storage: SQLite is the default database. A practical development machine should have enough disk space for dependencies, imported historical rows and saved simulation records. A Windows laptop with 8 GB RAM is a reasonable recommended setup, not a measured minimum.

2.4 Design and Implementation Constraints

- Budget and data: the implementation uses open source libraries and a public historical dataset. The imported date range and selected stocks define which simulations can be run.
- Security: passwords are stored as bcrypt hashes. Signed JWT tokens identify users, and backend ownership checks restrict access to each user's simulations. Logout removes the browser token. Server-side revocation and token expiry are not implemented in the current version.
- Design: page components, API access, trading rules, database models and machine learning code are kept in separate files. Prices use fixed-point database values and cash is rounded to two decimal places. Daily OHLC data limits the precision of simulated order execution.

2.5 Assumptions and Dependencies

- Assumptions: learners understand that the money is virtual and that historical outcomes do not guarantee future results. The local database remains available while the application is running. Simulation time is controlled by the saved session date, rather than the device clock.
- Dependencies: Internet access is needed to install packages and download the dataset, unless local copies are supplied. After import, the core local simulation does not depend on a live market feed or continuous access to the dataset host.

3. System Features and Functional Requirements

The following requirements describe the working application. Each requirement has a unique identifier so that implementation and verification can be traced to a specific feature.

3.1 Module 1: Accounts and Simulation Access

Req ID	Feature Description	Input / Validation Rule	Priority
FR-1.1	Register a learner account	Valid email; password length from 8 to 72 characters; duplicate email is rejected.	High
FR-1.2	Authenticate an existing account	Email and password are checked against the stored bcrypt hash.	High
FR-1.3	Protect personal simulation data	A valid signed token and matching session ownership are required.	High
FR-1.4	Create a simulation	Choose a starting date, positive cash balance and optional trading costs. Non-trading dates move to the next available date.	High
FR-1.5	End and reopen a saved review	Ending a session blocks new trades and date advances; saved results remain readable.	High

3.2 Module 2: Historical Data, Trading and Machine Learning

Req ID	Feature Description	Input / Validation Rule	Priority
FR-2.1	Import reproducible historical data	Import 12 stocks and NIFTY 50 for 2019 to 2021 from a pinned revision or local dataset folder.	High
FR-2.2	Validate imported records	Reject missing columns, null values, duplicate dates, invalid OHLC bounds and inadequate coverage.	High
FR-2.3	Restrict revealed information	Market queries, model inputs and journal outcomes shall not reveal data after the simulation date.	High
FR-2.4	Place virtual market and limit orders	Validate stock, Buy/Sell side, positive integer quantity, active session and positive limit price when required.	High
FR-2.5	Reserve cash and shares	Pending buys reserve estimated value plus charges; pending sells reserve shares. Reject insufficient availability.	High

Req ID	Feature Description	Input / Validation Rule	Priority
FR-2.6	Cancel a pending order	Only a pending order owned by the learner can be cancelled.	Medium
FR-2.7	Advance historical time	Reveal the next available trading day or a selected group of trading days. Stop when no later data exists.	High
FR-2.8	Execute eligible orders	Market orders fill at a later open. Limits use that day's open and high/low bounds. Same-day execution is excluded.	High
FR-2.9	Maintain average-cost holdings	Update quantity and average cost on purchases; calculate realised P&L on sales. Partial sales preserve average cost.	High
FR-2.10	Account for charges and corporate actions	Apply enabled simulated fees; credit dividends and apply imported split ratios during date advances.	High
FR-2.11	Save equity snapshots	Persist cash and close-marked equity for every revealed trading day.	High
FR-2.12	Prepare historical features	Calculate six return, volatility, range and volume features from observed raw OHLCV only. Exclude split-crossing windows.	High
FR-2.13	Train models automatically	When a signal is requested, train on available history if no run exists for that session, stock, version and date. Require 160 labelled examples.	High
FR-2.14	Evaluate the direction classifier	Use older 80% for evaluation training and later 20% for holdout testing, with three earlier walk-forward checks and a majority baseline.	High
FR-2.15	Save and reuse a model run	Persist preprocessing statistics, weights, cutoff, sample counts, test metrics and held-out predictions.	High
FR-2.16	Return a next-day direction score	Apply sigmoid; classify Up at probability 0.5 or above. Explain missing history and model quality.	High
FR-2.17	Personalize decision feedback	Use resolved one-day journal calls for the same stock and simulation to report correct Up/Down calls. Do not claim these records retrain market weights.	Medium

3.3 Module 3: Learning, Review and Visualization

Req ID	Feature Description	Input / Validation Rule	Priority
FR-3.1	Show stock and portfolio charts	Display historical candles and volume, equity snapshots, cash, allocation and risk measures.	High
FR-3.2	Record a decision before its outcome	Save stock, reason, Up/Down prediction and a 1, 5 or 20-day horizon; validate the reason and optional linked order.	High
FR-3.3	Freeze comparable model calls	For a one-day journal entry, save available model and baseline predictions. Later model updates shall not replace the original record.	High
FR-3.4	Resolve journal outcomes	Reveal the actual direction only after the required number of later stock closes is available.	High
FR-3.5	Review closed positions	When all holdings are sold, display closed-position status, cash balance and completed sale history.	High
FR-3.6	Compare decisions in Results	Show personal calls, saved model calls and actual outcomes; compare model and baseline on the same one-day records.	High
FR-3.7	Compare portfolio performance	Calculate return, NIFTY 50 comparison, maximum drawdown, available volatility and post-trade price movement.	High
FR-3.8	Provide learning and a challenge	Offer short lessons and a 20-trading-day challenge. Freeze its completed debrief at the challenge endpoint.	Medium
FR-3.9	Maintain a personal watchlist	Add or remove imported stocks for the authenticated user.	Medium
FR-3.10	Show historical context and ranking	Display curated events only through the revealed date; group leaderboard entries by identical simulation starting date.	Medium

3.4 Model Training and Decision Feedback

MarketLab uses a binary logistic regression classifier implemented with NumPy. Its task is to estimate whether the next available trading day closes above the current close. The model does not estimate an exact price or trade return. An unchanged close belongs to the Down class.

The six inputs are the 1-day, 5-day and 10-day returns, 10-day return volatility, the daily high-low range relative to the close, and volume relative to its previous 20-day average. Historical examples are labelled

Up when the next observed close is higher. The current day has no known next-day label and is used only for prediction.

Training requires at least 160 valid labelled examples, normally 181 daily rows before exclusions. Features are standardized using statistics from the relevant training subset. Batch gradient descent runs for 360 epochs with learning rate 0.08 and L2 regularization 0.025. A sigmoid converts the weighted feature score into an Up probability.

The earlier 80% of labelled examples train the evaluation model; the later 20% form a chronological holdout. Three earlier expanding-window checks provide additional comparisons. The app reports accuracy, precision, recall, Brier score, confusion counts and loss history. It labels a model above baseline only when its final holdout and earlier checks meet the application's thresholds. After evaluation, a final model is fitted on all currently observed labels.

Training is automatic when Explore or Portfolio requests signals, when a specific model is viewed, or when a one-day journal entry needs a model snapshot. It runs during the request; there is no separate training scheduler. Saved runs are reused for the same session, stock, date and model version.

User-specific feedback uses past resolved one-day journal predictions for the same stock and simulation. It reports directional accuracy and the learner's tendency to choose Up or Down. At least five resolved calls are needed for pattern feedback. The current classifier is trained on OHLCV; the personal history does not retrain its weights or change the market probability.

3.5 Demonstration and Acceptance Flow

Create the Buying the Bottom scenario on 23 March 2020 with Rs. 1,00,000 and leave the demo-portfolio option unchecked. Research RELIANCE and record a one-day prediction. Place a market buy for 10 shares and confirm that it is Pending. Advance one trading day and check the fill, cash deduction, holdings and resolved journal outcome. Later, sell the shares and advance again. After ending the session, verify the final cash, closed-position message, sale history and Results comparison. Profit is an observed outcome, not an acceptance condition.

4. External Interface Requirements

4.1 User Interfaces (UI)

The interface provides a login screen, simulation setup, an overview, stock research, replay, portfolio, Model Insights, a decision journal, results and learning pages. Forms display validation errors and action feedback. Completed simulations show final balances and a new-simulation option. A closed portfolio displays sale history instead of suggesting that no trading has taken place.

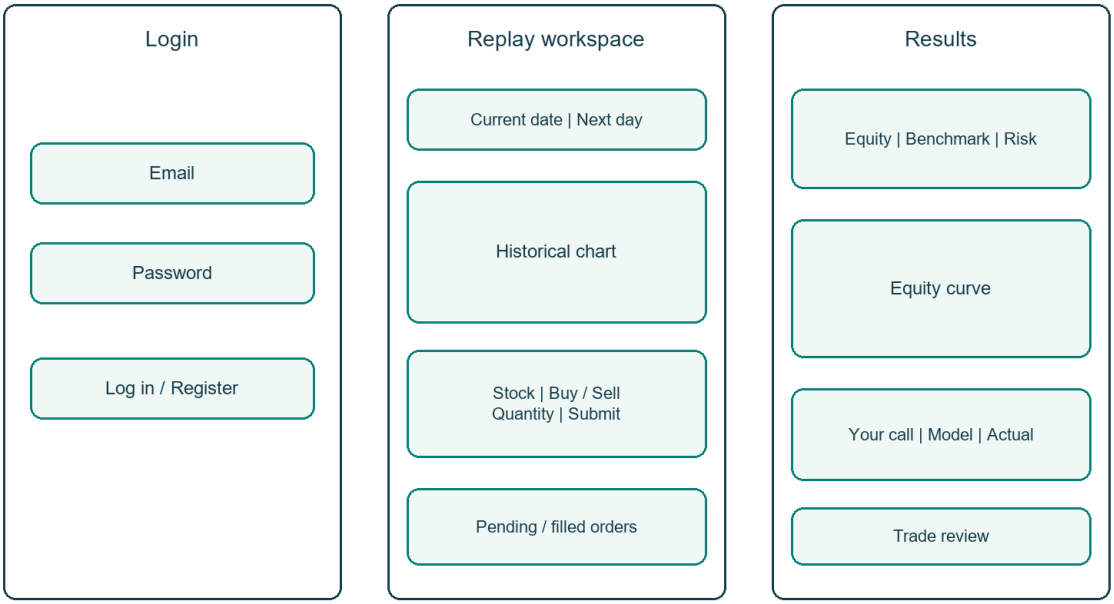


Figure 3. Simplified wireframes of login, replay and results screens.

4.2 Hardware Interfaces

MarketLab uses ordinary computer input and display devices.

- Standard peripherals: keyboard, mouse or touch input, and a browser display. Charts support stock selection and interactive navigation.
- No sensors, embedded controllers or dedicated trading hardware are required. The model runs on the CPU.

4.3 Software Interfaces

- Browser to server: Next.js and React call FastAPI endpoints through a shared fetch helper. Pydantic validates inputs. SQLAlchemy connects the backend to SQLite.

- Data and charts: Pandas and PyArrow read historical Parquet data, NumPy trains the model, Lightweight Charts renders stock candles and volume, and Recharts renders portfolio and evaluation graphs. Lucide React provides interface icons.

4.4 Communication Interfaces

- Local traffic: the frontend normally runs on localhost:3000 and the API on localhost:8000. The launcher can select free ports when these are occupied. Local development uses HTTP. A public deployment would require an HTTPS configuration.
- Requests and responses generally use UTF-8 JSON. Login uses form-encoded credentials. Authenticated requests carry a Bearer token in the Authorization header. CORS allows the configured localhost development origins.

5. Non-Functional Requirements (NFRs)

Attribute	Requirement / Acceptance Criterion	Verification Method
5.1 Performance	Ordinary pages shall provide loading feedback. Model results for an unchanged stock, date and version shall reuse the saved run. Cold training time must be measured separately; no concurrency target has been verified.	Repeat requests and inspect saved run IDs; measure local response times before assigning a performance target.
5.2 Security	Passwords shall be hashed; invalid authentication shall be rejected; one learner shall not access another learner's sessions, orders or journal. Token expiry, revocation and rate limiting remain future hardening work.	Authentication and ownership tests, database inspection and code review.
5.3 Reliability	Saved records shall survive a normal restart. Cash and holdings shall stay consistent with fills. Unique fill and snapshot keys shall help prevent duplicate execution.	Trading engine tests, restart checks and inspection of fills, ledger entries and snapshots.
5.4 Usability and portability	Forms shall explain failures and show order state. Completed-session screens shall provide suitable navigation. Layouts shall adapt to common desktop and mobile widths.	Browser walkthroughs and viewport checks; broader cross-browser testing remains to be completed.
5.5 Numerical correctness	Cash shall be rounded to paise; holdings shall use average-cost accounting; returns and risk measures shall use saved observations.	Tests for fills, partial sales, charges and portfolio totals.
5.6 ML reproducibility	Identical observed inputs shall produce repeatable training. Feature and test data shall follow chronological cutoffs. Insufficient history shall not produce a trained model.	Feature, deterministic training, chronological evaluation and API cutoff tests.

6. System Design and Analysis Models

The diagrams below describe the current design. The context and data flow diagrams show the application boundary and main processes. The relationship diagram summarizes the main database links. The sequence diagram follows a buy order from submission to its next-day fill.

Model	Purpose	Included Figure
Context / architecture	Show the learner, historical data, application layers and storage.	Figures 1 and 2
Use cases	Show the registered learner's principal actions.	Figure 4
Data flow	Show authentication, price retrieval, trading, modeling and review.	Figure 5
Entity relationships	Show primary data entities and selected key relationships.	Figure 6
Sequence	Follow an order from submission to its next-day fill.	Figure 7

6.1 Learner Use Cases

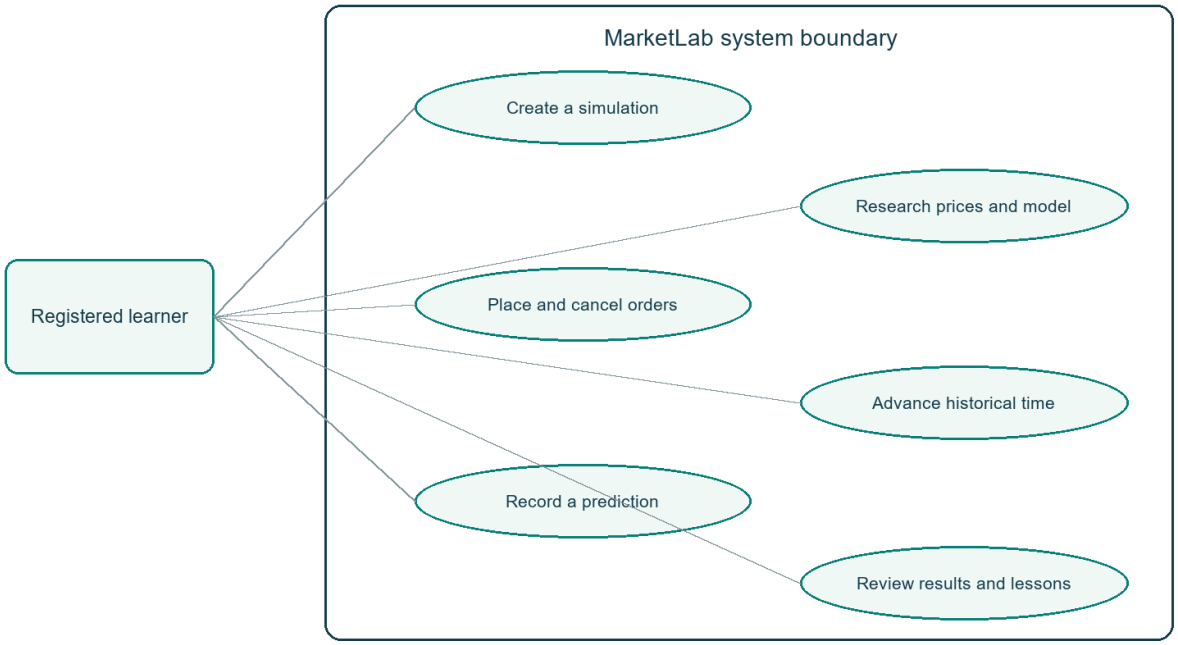


Figure 4. Main interactions available to a registered learner.

6.2 Data Flow

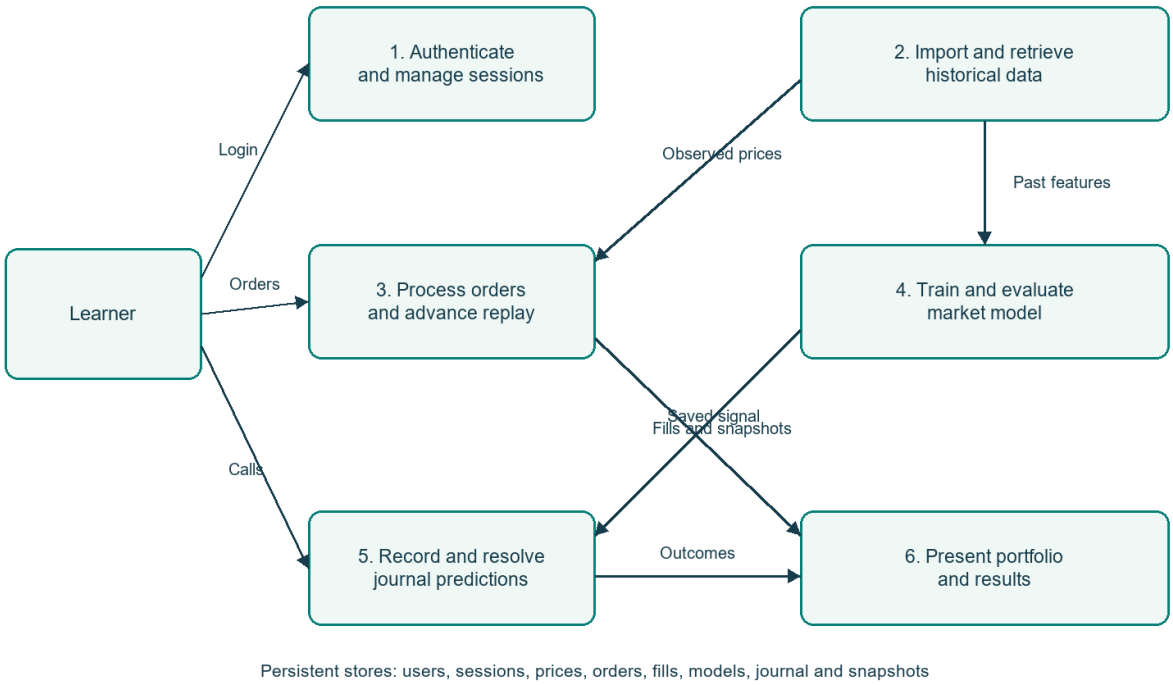


Figure 5. Level 1 view of the principal application processes.

6.3 Entity Relationships

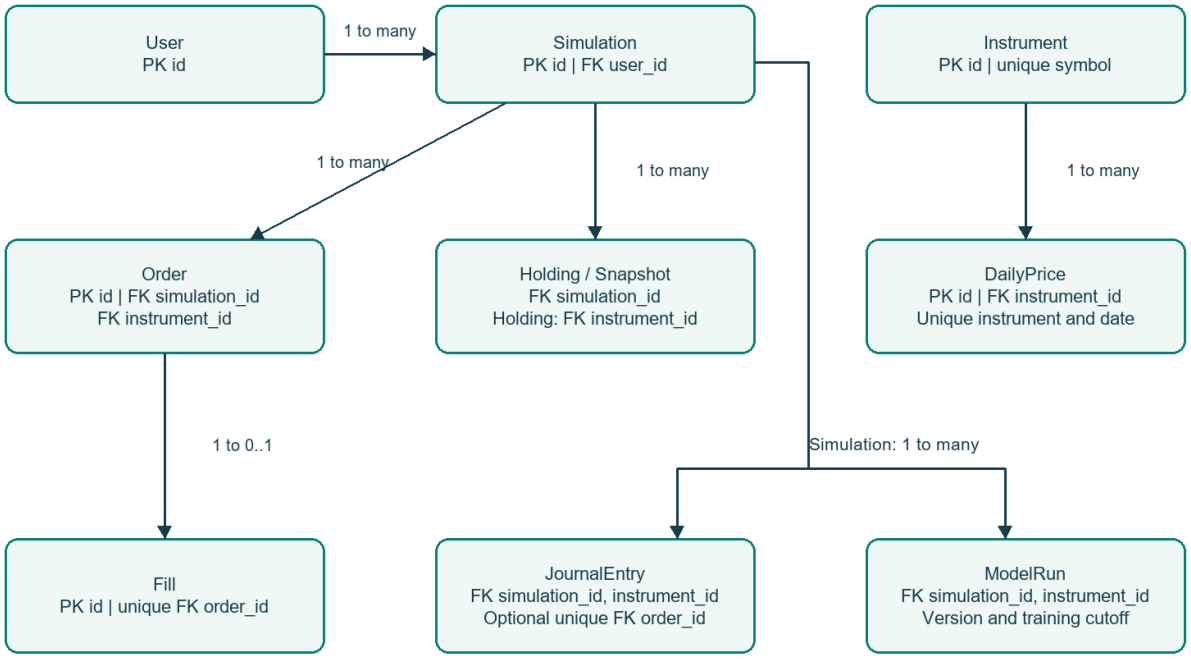


Figure 6. Simplified relationship map of the main database entities.

6.4 Buy Order Sequence

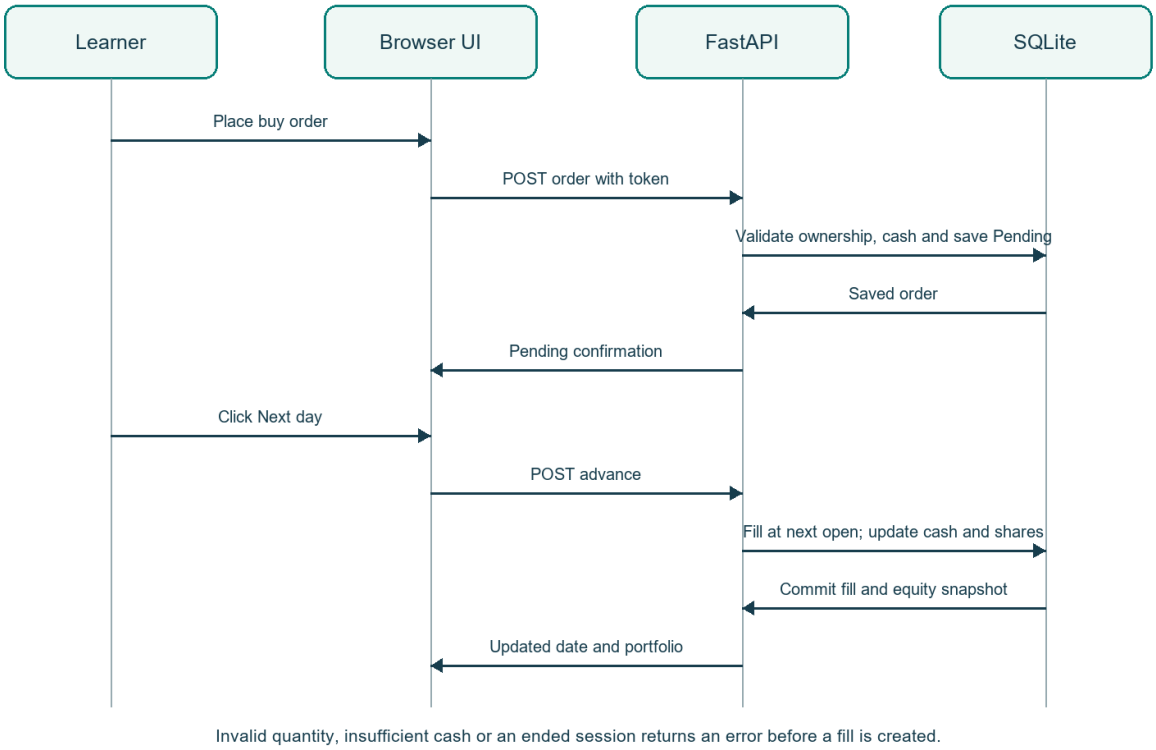


Figure 7. A market order is queued first and executed on a later trading day.

Database structure: prices are separated from instruments, orders from their fills, and simulations from per-day snapshots. A model run has a unique session, stock, version and cutoff combination. A fill has a unique order link. A holding has a unique session and stock combination. Ledger entries retain cash movements for review. Figure 6 is a summary and does not draw every foreign key.

Sequence exceptions: an invalid token, an order outside the learner's session, an invalid quantity, insufficient available cash or an ended simulation produces a validation error. A market order may be rejected at execution if the actual opening cost exceeds available cash. A limit order remains pending until its condition is met or the learner cancels it.

6.5 Verification Status

At preparation, all 11 backend tests passed and the frontend production build completed with TypeScript checks. Existing tests cover next-day fills, average cost, partial sales, limits, reservations, ownership, model feature cutoffs, chronological evaluation, repeatable training, journal timing and challenge behaviour. These checks do not establish load capacity, certified security or future market performance.

7. Academic Review and Sign-Off

Prepared by Alok Srivastava for review under the guidance of Mr. Faizan. Signatures and review dates are to be entered by the relevant academic reviewers after evaluation. No approval is implied by this document.

Project Guide	Internal Examiner	Head of Department
Mr. Faizan	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____